

the former into our project. The Pandas library is used for this purpose. We bring the data set into the Pandas data frame using the following line of code:

```
sentiment_dataframe = pd.read_csv(
    "content/drive/MyDrive/Data/sentiments -
    sentiments.tsv", sep = '\t')
```

Now that we have the data set in our project, let us manipulate it so that our algorithm can understand the features better. We begin by giving names to our columns in the data set. This is done by using the line of code given below:

```
sentiment_dataframe.columns =
    ["label", "body_text"]
```

We then assign numerical labels to the classes — negative is replaced with 1 and positive is replaced with 0. Figure 2 shows how the data frame looks at this stage.

The next step is the preprocessing of the data. This is a very important step as it helps us to convert string/text data

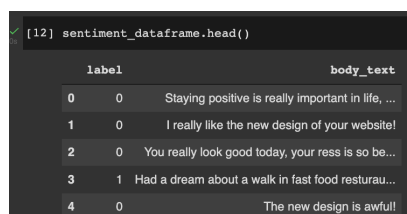
into numerical data (machine learning algorithms can understand/process numerical data and not text). Also, the redundant and useless data needs to be removed as it may taint our training model. We remove the noisy data, missing values and other non-consistent data in this step.

We will add the features text length and punctuation count in the data frame specifically for this application. We will also do the stemming, i.e., we will convert all similar words (like 'give', 'giving', etc) into a single form. Once this is done, we divide the data set into two -- X and Y -- where X is the *features* and Y is the *prediction* class.

This is done using the following piece of code. Figure 3 shows the data frame after these steps are taken.

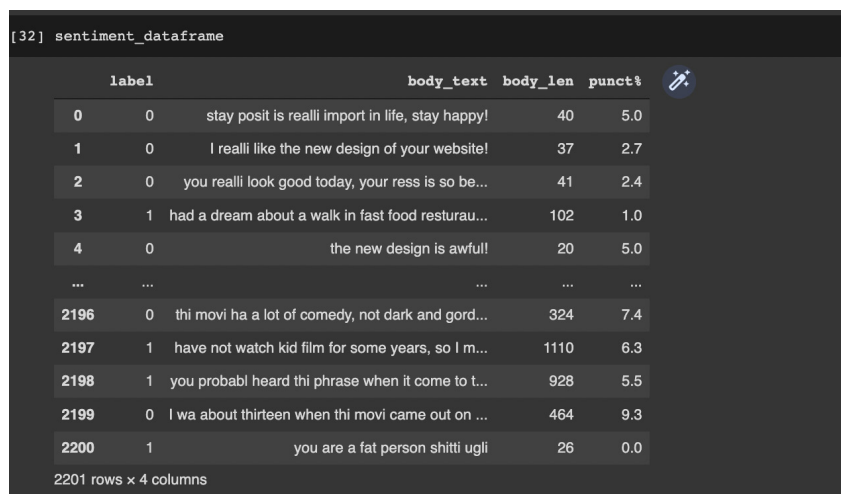
```
def count_punct(text):
    count = sum([1 for char in text if
        char in string.punctuation])
    return round(count/(len(text) -
        text.count(" ")),3)*100
```

```
tokenized_tweet = sentiment_
dataframe['body_text'].apply(lambda x:
    x.split())
stemmer = PorterStemmer()
tokenized_tweet = tokenized_tweet.
apply(lambda x: [stemmer.stem(i) for i
    in x])
```



	label	body_text
0	0	Staying positive is really important in life, ...
1	0	I really like the new design of your website!
2	0	You really look good today, your res is so be...
3	1	Had a dream about a walk in fast food restau...
4	0	The new design is awful!

Figure 2: Data frame with basic modifications



	label	body_text	body_len	punct%
0	0	stay posit is realli import in life, stay happy!	40	5.0
1	0	I realli like the new design of your website!	37	2.7
2	0	you realli look good today, your res is so be...	41	2.4
3	1	had a dream about a walk in fast food restau...	102	1.0
4	0	the new design is awful!	20	5.0
...	...	...	...	...
2196	0	thi movi ha a lot of comedy, not dark and gord...	324	7.4
2197	1	have not watch kid film for some years, so I m...	1110	6.3
2198	1	you probabl heard thi phrase when it come to t...	928	5.5
2199	0	I wa about thirteen when thi movi came out on ...	464	9.3
2200	1	you are a fat person shitti ugli	26	0.0

2201 rows x 4 columns

Figure 3: Data frame after the division of the data set

```
for i in range(len(tokenized_tweet)):
    tokenized_tweet[i] = '
'.join(tokenized_tweet[i])
sentiment_dataframe['body_text'] =
    tokenized_tweet
sentiment_dataframe['body_len'] =
    sentiment_dataframe['body_text'].
    apply(lambda x: len(x) - x.count(" "))
sentiment_dataframe['punct%'] =
    sentiment_dataframe['body_text'].
    apply(lambda x: count_punct(x))
X = sentiment_dataframe['body_text']
y = sentiment_dataframe['label']
```

We now need to convert the string into numerical data. We use a count vectorizer for this purpose; that is, we get the counts of each word and convert it into a vector.

After this, features such as length of text and punctuation count in the dataframe, i.e., X, are calculated. A sample of X is shown in Figure 4.

Now the data is ready for training. The next step is to determine which algorithms we are going to use for training our model. As has been mentioned before, we are going to try several algorithms and determine the best one for sentiment analysis. Since we are basically trying to do binary classification, the following algorithms can be used:

- K-nearest neighbors (KNN)
- Logistic regression
- Support vector machines (SVMs)
- Stochastic gradient descent
- Naive Bayes
- Decision tree
- Random Forest

We first need to split our data set into testing and training data. This is done by using the *sklearn* library using the following code:

```
from sklearn.model_selection import
    train_test_split
X_train, X_test, y_train, y_test =
    train_test_split(X,y, test_size = 0.20,
        random_state = 99)
```

We will use 20 per cent of the data for testing and 80 per cent for the